AgentsGate × Claude Code / OpenClaw

Test Procedure Guide

Verifying AgentsGate with Claude Code and OpenClaw — English edition (the Japanese edition is agentsgate-claude-code-test-guide_jp.pdf)

Version 1.2 · 2026-07-29

Contents

1. What this document is for

Protection levels

A score alone cannot say what should happen. DROP TABLE scores 1.00 and SELECT * FROM users scores 0.60 — they differ in kind, not degree, so moving the threshold until the SELECT passes also clears DELETE FROM orders at 0.90. Every built-in rule therefore carries a category, and the protection level says what to do with each.

agentsgate level — show what is stopped right now, and why

agentsgate level strict — change it. The dashboard has the same switch in its header, and changing it there applies to the running proxy immediately.

minimal — only wholesale destruction is stopped: DROP, TRUNCATE, a DELETE with no WHERE. For a scratch project.

balanced — the default. Adds credential files (.env, .pem) and outbound deletions. Writing code, running tests and editing rows all run without a prompt.

strict — adds personal-data reads, outbound sends, shell commands and deletions to what needs a human. For data that is not only yours.

balanced is a convenience posture, not a data-protection one: shell commands run and a table called users can be read. Policy rules are applied after the level and override it.

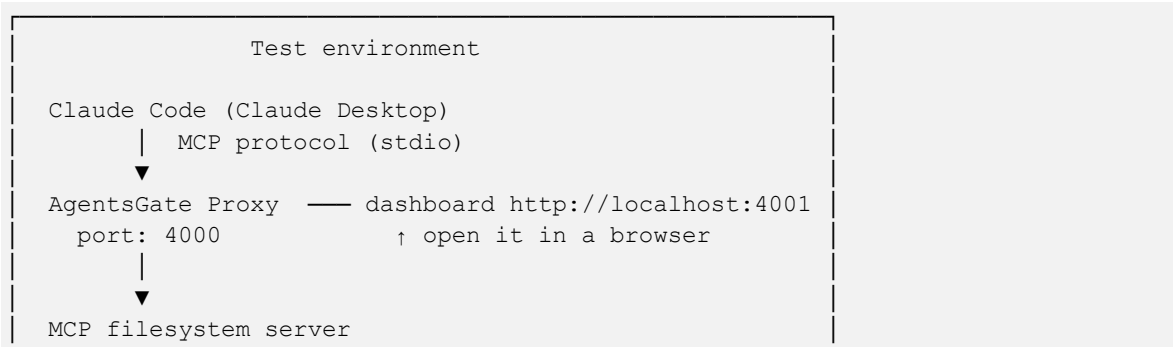
This is a procedure for testing AgentsGate through Claude Code and OpenClaw. You will drive real operations and confirm that AgentsGate intercepts, records, scores and blocks MCP tool calls as it should. Sections 1–7 cover the basic scenario with Claude Code; section 8 is an advanced scenario combining Docker and OpenClaw.

Audience: developers and operators who already have AgentsGate installed.

Time: about 30 minutes for sections 1–7, about 50 minutes including section 8.

2. The test environment

There are two setups here: the basic one with Claude Code in section 4, and the Docker + OpenClaw one in section 8.



```
│  (runs the actual tool)                                  │
│                                                          │
│  DB: ~/.agentsgate/agentsgate.db  (SQLite)               │
└──────────────────────────────────────────────────────────┘
```

💡 For the advanced setup (Docker × OpenClaw × AgentsGate), see the diagram in 8.1.

Component (basic setup — section 4)	Detail
Claude Code	The MCP client (agent) that ships with Claude Desktop
AgentsGate Proxy	localhost:4000 — intercepts, records and scores MCP operations
AgentsGate Dashboard	localhost:4001 — web UI for reading the operation log
MCP filesystem server	The server Claude uses for the actual file operations
SQLite DB	~/.agentsgate/agentsgate.db — where every log is persisted

3. Pre-flight checklist

Check all of the following before you begin.

Check	How to check	Result
Node.js 20 or later is installed	`node --version`	☐ OK ☐ NG
The agentsgate command works	`agentsgate --version`	☐ OK ☐ NG
AgentsGate is running	`agentsgate status`	☐ OK ☐ NG
Claude Desktop is installed	Check your applications list	☐ OK ☐ NG
http://localhost:4001 opens in a browser	The dashboard should load	☐ OK ☐ NG
Docker Desktop is installed (section 8 only)	docker --version	□ OK □ NG
OpenClaw is installed (section 8 only)	openclaw --version	□ OK □ NG

4. The test, step by step

4.1 Confirming AgentsGate is running

Run this in a terminal to confirm AgentsGate is up.

```
agentsgate status
```

Expected output:

```
AgentsGate is RUNNING
  PID:       27224
  Proxy:     http://localhost:4000
  Dashboard: http://localhost:4001
  Started:   2026-03-23T09:07:53.176Z
  Database:  C:\Users\<username>\.agentsgate\agentsgate.db
```

If it says STOPPED, start it:

```
agentsgate start
```

The defaults are 4000 for the proxy and 4001 for the dashboard — the dashboard is always the proxy port plus one. If a port is taken and startup fails, pass another as a positional argument, e.g. agentsgate start 8080 (dashboard on 8081).

4.2 Injecting AgentsGate into Claude Desktop

Adding AgentsGate to Claude Desktop's MCP configuration routes every MCP tool call Claude Code makes through AgentsGate.

⚠ *Quit Claude Desktop completely before running this command.*

```
agentsgate inject
```

On success you will see:

```
Injected AgentsGate proxy into 1 server(s):
  + filesystem

Config:  ~/Library/Application Support/Claude/claude_desktop_config.json
Backup:  ~/Library/Application
Support/Claude/claude_desktop_config.agentsgate.bak.json

Restart Claude Desktop to apply changes.
```

Restart Claude Desktop. To check the injection state:

```
agentsgate inject status
```

Expected output:

```
Claude Desktop MCP server injection status
(C:\Users\<username>\AppData\Roaming\Claude\claude_desktop_config.json):

  [injected]   filesystem              npx -y @modelcontextprotocol/server-
filesystem C:\work
```

A backup of the original config is written automatically; eject restores it.

4.3 Calling MCP tools from Claude Code

Start Claude Desktop and open a Claude Code session, then ask Claude for each of the following to trigger MCP tool calls.

[Test 1] Listing files (low risk)

```
Ask Claude: "List the folders directly under C:¥Users."
```

What should happen:

- AgentsGate intercepts the directory listing
- The risk score is low (< 0.30), so the verdict is allow
- Claude answers with the folder list

[Test 2] Reading a file (medium risk)

> Ask Claude: "Check whether the file C:¥Users¥<username>¥.agentsgate¥agentsgate.db exists."

What should happen:

- AgentsGate intercepts and records the read
- A risk score is calculated
- The operation appears in the dashboard

[Test 3] Creating a file (high risk)

> Ask Claude: "Create the file C:¥Users¥<username>¥Desktop¥test-agentsgate.txt and write 'AgentsGate test' into it."

What should happen:

- AgentsGate intercepts the write
- The score is 0.30 or higher, so the verdict is require_approval or block
- Claude cannot complete the operation, or it waits for your approval

4.4 Checking the operation log in the dashboard

Open the AgentsGate dashboard in a browser:

```
http://localhost:4001
```

Confirm all three tests appear in the operation list on the Overview tab.

Check	Expected	Result
Test 1 (file listing) is recorded	action: allow, riskScore < 0.30	☐ OK ☐ NG
Test 2 (file read) is recorded	action: allow or require_approval	☐ OK ☐ NG
Test 3 (file write) is recorded	action: require_approval or block	☐ OK ☐ NG
Every operation carries an agentId	e.g. claude-desktop	☐ OK ☐ NG
Every operation carries a	The execution time is correct	☐ OK ☐ NG

timestamp		

The same thing from the CLI:

```
# list recent operations
agentsgate ops tail --limit=20

# blocked operations only
agentsgate ops tail --action=block
```

4.5 Checking risk scores and blocking

Click each operation's row on the Overview tab to see its detail.

What to look at:

- riskScore — the risk score, 0.0 to 1.0
- decision — allow / require_approval / block
- firedRules — the risk rules that matched
- operation.params — the operation parameters, with sensitive values shown as [REDACTED]

The same detail from the CLI:

```
# list recent risk assessments
agentsgate risk

# risk detail for one operation id
agentsgate explain <operationId>
```

Sample explain output:

```
Operation Explanation — op-7f3a91c4

  Agent:     claude-desktop
  Tool:      filesystem · write_file
  Session:   sess-2f9c
  Timestamp: 2026-07-29T09:14:22.104Z

Decision: BLOCK (risk 72.0%)

Fired Risk Rules:
  [L1] L1_OVERWRITE_FILE              ████████░░░░░░░░ 50%
  [L1] L1_SENSITIVE_PATH_WRITE        ███░░░░░░░░░░░░░ 22%
```

4.6 Reading the debug information

If something fails during the test, or you want to see what is happening internally, use these.

Method 1: the agentsgate errors command

Show the recent error history:

```
agentsgate errors

# to choose how many
agentsgate errors 20
```

When there are none:

```
No errors recorded.
```

Sample output when there are:

```
TIMESTAMP                  MODULE    MESSAGE
─────────────────────────  ────────  ──────────────────────────────────────
2026-03-23T09:15:22.123Z   proxy     Connection refused: upstream MCP server
```

Method 2: debug mode (AGENTSGATE_DEBUG=1)

When you need a full stack trace, restart AgentsGate in debug mode. Every error then prints its detail to stderr as it happens.

Stop AgentsGate, then restart it in debug mode:

```
agentsgate stop

# Windows (Command Prompt)
set AGENTSGATE_DEBUG=1 && agentsgate start

# macOS / Linux / Git Bash
AGENTSGATE_DEBUG=1 agentsgate start
```

Sample debug output:

```
[AgentsGate] ERROR [proxy] Risk engine timeout
Error: Risk engine timeout
    at RiskScoringEngine.assess (dist/cli.js:87:13)
    at evaluateRisk (dist/cli.js:192:24)
  operationId: 3fa85f64-5717-4562-b3fc-2c963f66afa6
  context: {"tool":"file_system","method":"write"}
```

Method 3: fetching the errors over the REST API

The dashboard API returns the same list as JSON:

```
curl http://localhost:4001/errors?limit=10
```

Sample response:

```
{
  "errors": [
    {
      "id": "3fa85f64-...",
      "timestamp": "2026-03-23T09:15:22.123Z",
      "module": "proxy",
      "message": "Connection refused",
      "operationId": "op-abc123"
    }
  ],
  "total": 1
}
```

Method 4: health and self-check

Check the overall state:

```
# health
agentsgate health

# whole-installation self-check
agentsgate doctor
```

Sample doctor output when everything is healthy:

```
AgentsGate Doctor

  ✓ Config file               port=4000 dashboard=4001
  ✓ Policy file               0 rule(s) loaded
  ✓ State directory           C:\Users\<username>\.agentsgate
  ✓ SQLite database           agentsgate.db (accessible, empty)
  ✓ Shadow git repo           shadow-repo
  ✓ Proxy process             running (PID 27224)
  ✓ Claude Desktop config     1 server(s), 1 injected

All checks passed.
```

Method 5: detecting tampering in the operation log

Set audit.signingSecret in config.json and every operation log entry is signed. Each signature covers the signature of the entry before it, so this detects not only a record being edited but also one being deleted from the middle.

{ "audit": { "signingSecret": "a long random string" } }
After the test, run this to confirm nothing has been tampered with:

agentsgate verify-logs
Chain: intact means nothing was edited or removed. If the chain is broken, the output says which record it breaks at.

⚠ Deleting the newest records is not detected — that leaves a shorter but internally consistent chain.

4.7 Checking checkpoints and rollback

AgentsGate saves a checkpoint (a snapshot) before any operation scoring 0.30 or higher. If test 3 (the file write) came out as require_approval, a checkpoint should exist.

List the checkpoints:

```
agentsgate checkpoints
```

Sample output:

```
ID                                      CREATED                     OPS
──────────────────────────────────────  ──────────────────────────  ────

cp-abc123...                            2026-03-23T09:20:11.000Z    1
```

To roll back to a particular checkpoint:

```
agentsgate rollback <checkpointId>
```

⚠ *Rollback really does restore files. Run it in a test environment.*

5. Test checklist

Tick every item below once the test is done.

Category	Check	Result
Startup	agentsgate status reports RUNNING	☐ OK ☐ NG
Startup	The dashboard opens at http://localhost:4001	☐ OK ☐ NG
Startup	agentsgate doctor prints All checks passed	☐ OK ☐ NG
Injection	agentsgate inject status shows the target server as [injected]	☐ OK ☐ NG
Logging	Claude Code's operations appear in the dashboard	☐ OK ☐ NG
Logging	Operations carry agentId, timestamp and riskScore	☐ OK ☐ NG
Risk scoring	The file listing (low risk) comes out as allow	☐ OK ☐ NG
Risk scoring	The file write (high risk) comes out as require_approval or block	☐ OK ☐ NG
Risk scoring	agentsgate explain shows the fired rules	☐ OK ☐ NG
Debugging	agentsgate errors shows the error history	☐ OK ☐ NG
Debugging	AGENTSGATE_DEBUG=1 prints stack traces to stderr	☐ OK ☐ NG
Debugging	GET /errors returns JSON	☐ OK ☐ NG
Checkpoints	agentsgate checkpoints lists the snapshots	☐ OK ☐ NG
Audit	agentsgate audit lists the past operation log	☐ OK ☐ NG
Docker / OpenClaw (§8)	The Docker container (agentsgate-pg) is running	□ OK □ NG
Docker / OpenClaw (§8)	agentsgate inject-pg succeeded	□ OK □ NG
Docker / OpenClaw (§8)	OpenClaw's list_tables comes out as allow (riskScore 0.05)	□ OK □ NG
Docker / OpenClaw (§8)	OpenClaw's SELECT comes out as allow (0.05) and a write as require_approval	□ OK □ NG
Docker / OpenClaw (§8)	OpenClaw's DROP TABLE comes out as block (L1_DB_DROP)	□ OK □ NG

6. Cleaning up afterwards

Put the environment back as it was:

1. Delete the files you created during the test (by hand)
2. Prune old logs (optional):

```
agentsgate ops prune --older-than=86400000   # delete logs older than 24 hours
```

3. Undo the Claude Desktop injection (optional):

```
# close Claude Desktop first
agentsgate eject
```

4. Stop AgentsGate (optional):

```
agentsgate stop
```

> 💡 *If you are going to keep using AgentsGate, you do not need eject or stop.*

💡 If you ran the section 8 scenario, also follow the cleanup in 8.6 (inject-pg remove, docker stop).

7. Troubleshooting

No operations appear in the dashboard

- Check the injection is active with agentsgate inject status
- Check you restarted Claude Desktop after injecting
- Check the proxy is running with agentsgate status
- Check every item passes with agentsgate doctor

agentsgate inject fails

- Check Claude Desktop has fully quit, including the tray icon
- Check claude_desktop_config.json is writable
- Try running the terminal as administrator

Everything comes out as allow

- Check the injection took with agentsgate inject status
- Check Claude Desktop has finished restarting
- Check with agentsgate config that the intervention thresholds are not set unexpectedly high

agentsgate errors shows errors

- Start with AGENTSGATE_DEBUG=1 and read the stack trace
- Use agentsgate doctor to find what is wrong
- Use the module column to see which component the error came from

Claude Code says it cannot use the tools

- Check the proxy is running with agentsgate status
- Check claude_desktop_config.json is correct with agentsgate inject status
- Restart Claude Desktop completely

Docker / OpenClaw problems

- See section 8.7.

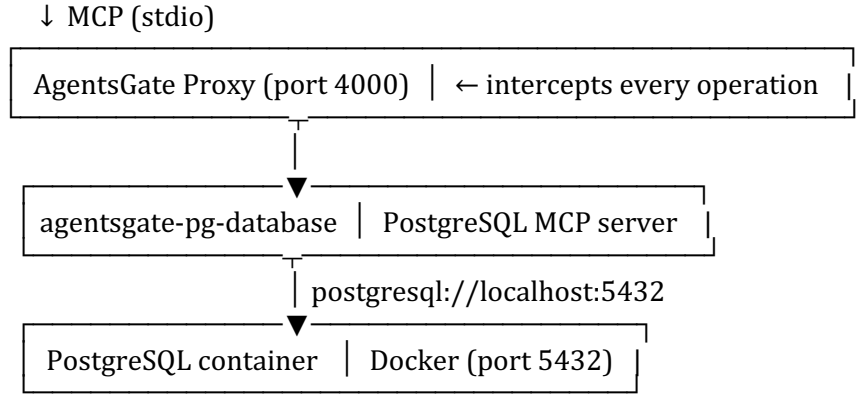
---

8. Advanced scenario — Docker × OpenClaw × AgentsGate

This scenario points an OpenClaw agent at a PostgreSQL database running in Docker, with AgentsGate in between. You confirm that every SQL operation is intercepted, scored and recorded, and that dangerous ones are blocked or held for approval.

Time: about 20 minutes. Assumes: Docker Desktop is installed.

8.1 How it fits together



Component	Detail
OpenClaw agent	MCP client — calls tools through AgentsGate's stdio proxy
AgentsGate Proxy	localhost:4000 — intercepts, records and scores MCP operations
agentsgate-pg-database	The PostgreSQL MCP server that ships with AgentsGate
PostgreSQL (Docker)	localhost:5432 — the test database container

8.2 Setting up

Step 1 Start PostgreSQL in Docker

Start the PostgreSQL container:

```
docker run -d \
  --name=agentsgate-pg \
  -e POSTGRES_USER=testuser \
  -e POSTGRES_PASSWORD=testpass \
  -e POSTGRES_DB=testdb \
  -p 5432:5432 \
  postgres:16
```

Confirm it started — "healthy" or "running" is fine:

```
docker ps --filter name=agentsgate-pg
```

Step 2 Register the PostgreSQL MCP server

Close Claude Desktop, then run:

```
agentsgate inject-pg --connection-string=postgresql://testuser:testpass@localhost:5432/testdb
```

The success message:

```
[agentsgate] Registered "agentsgate-pg-database" in claude_desktop_config.json
  connection: postgresql://testuser:***@localhost:5432/testdb
```

Restart Claude Desktop / Claude Code for the change to take effect.

Step 3 Update the OpenClaw configuration

Open ~/.openclaw/openclaw.json and wrap the agentsgate-pg-database server so it goes through AgentsGate:

```
{
  "mcpServers": {
    "pg-database": {
      "command": "agentsgate",
      "args": [
        "proxy", "stdio",
        "--cmd", "node",
        "--args", "~/.npm-global/lib/node_modules/agentsgate/mcp-servers/pg-database/index.js,--
connection-string,postgresql://testuser:testpass@localhost:5432/testdb",
        "--agentId", "openclaw",
        "--tool", "pg-database"
      ]
    }
  }
}
```

💡 Run npm bin -g to find your global install path and adjust the path above to match.

Step 4 Start AgentsGate

```
agentsgate start
```

Confirm it started:

```
agentsgate status
```

8.3 The test operations

Give OpenClaw each of the following in turn and confirm AgentsGate handles it correctly.

[Test 1] Listing the tables (low risk)

Ask OpenClaw: "List the tables in the testdb database."

What should happen:

- list_tables is called; AgentsGate scores it 0.05 and allows it
- The operation is recorded on the dashboard's Overview tab
- OpenClaw answers with the table list

[Test 2] Running a SELECT (low risk) and a write (medium risk)

Ask OpenClaw: "SELECT the first 5 rows from every table in testdb." Reads are allowed. To see an approval gate, ask for a write instead — "update the status column in the users table" — which calls execute and scores 0.30, requiring approval.

What should happen:

- query is called; AgentsGate scores it 0.05 and allows it
- If you asked for a write, it appears in the pending-approval list on the Overview tab
- Once approved, OpenClaw answers with the result

[Test 3] Attempting DROP TABLE (high risk, blocked)

Ask OpenClaw: "Drop the users table in testdb."

What should happen:

- execute_ddl is called; AgentsGate scores it 1.00 and blocks it
- OpenClaw receives an error saying the operation was blocked
- The dashboard records it as block, with L1_DB_DROP in firedRules

8.4 Checking the results in the dashboard

Open the AgentsGate dashboard in a browser:

http://localhost:4001
Confirm all three tests appear in the operation list on the Overview tab. Click the row for test 3 and check:

- decision: block
- riskScore: 1.00
- firedRules: ["L1_DB_DROP"]
- tool: pg-database, agentId: openclaw

The same from the CLI:

agentsgate ops tail --limit=10
agentsgate ops tail --action=block

8.5 Checklist

Check every item below.

Check	Expected	Result
The Docker container is running	docker ps shows agentsgate-pg	□ OK □ NG
inject-pg succeeded	agentsgate inject status shows agentsgate-pg-database	□ OK □ NG
Test 1 (table listing) is recorded	action: allow, riskScore: 0.05	□ OK □ NG
Test 2 (SELECT) is allowed and a write waits for approval	action: allow (SELECT) / require_approval (write)	□ OK □ NG
Test 3 (DROP TABLE) is blocked	action: block, riskScore: 1.00, L1_DB_DROP fired	□ OK □ NG
Every operation carries agentId: openclaw	Check the Agent column in the dashboard	□ OK □ NG
agentsgate errors is empty	No errors recorded.	□ OK □ NG
agentsgate doctor returns All checks passed	All checks pass	□ OK □ NG

8.6 Cleanup

Put the environment back as it was:

5. Remove the PostgreSQL MCP server registration:

agentsgate inject-pg remove
6. Stop and remove the Docker container:

docker stop agentsgate-pg && docker rm agentsgate-pg
7. Put the OpenClaw configuration back:

Remove the pg-database entry from ~/.openclaw/openclaw.json, or restore what was there before you wrapped it with AgentsGate.

8. Stop AgentsGate (optional):

agentsgate stop

8.7 Troubleshooting

Cannot connect to the Docker container

- Check the agentsgate-pg container shows as "Up" in docker ps
- Check nothing else on the host is using port 5432 (netstat -an | grep 5432)
- Check the username, password and database name in the connection string

OpenClaw says it cannot use the tools

- Check the proxy is running with agentsgate status
- Check the agentsgate proxy path in ~/.openclaw/openclaw.json against npm bin -g

- Restart OpenClaw so it reloads the configuration

SELECT is being blocked

- Check the block threshold (intervention.blockAtOrAbove) with agentsgate config
- The default is 0.70. A SELECT goes through query and scores 0.05, so allow is correct. DELETE or UPDATE without a WHERE clause, and DDL, score 0.85 or higher and are blocked.
- You can also add a rule to policy.json that allows SELECT on the pg-database tool

Related documents

- Beginner's installation guide → docs/agentsgate-beginner-guide.pdf
- Policy configuration guide → docs/policy-guide.md
- REST API reference → docs/api-reference.md
- OpenClaw guide → docs/openclaw-user-guide.md